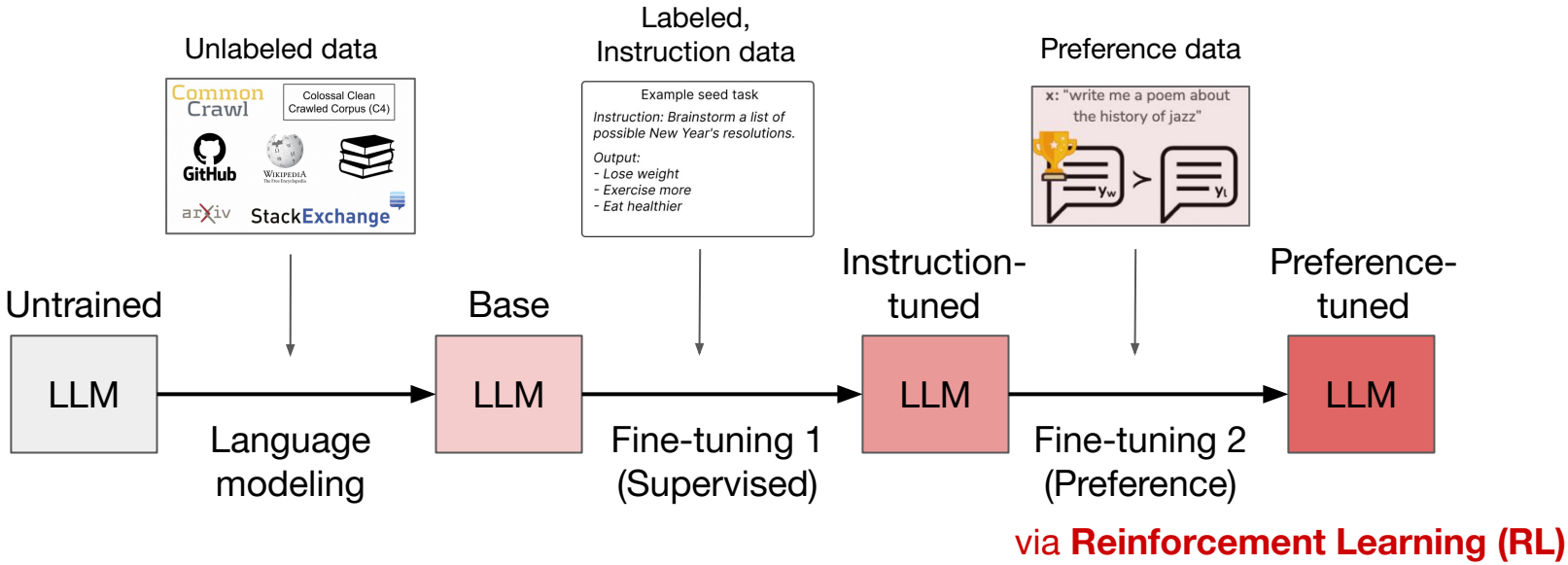
INFO-I590 Fundamentals and Applications of LLMs

RLHF

Reinforcement Learning from Human Feedback

Jisun An

Three steps of creating a high-quality LLM



Q: What is 1+1?

A: 1 + 1 + …

A: 2

A: The answer is 2.

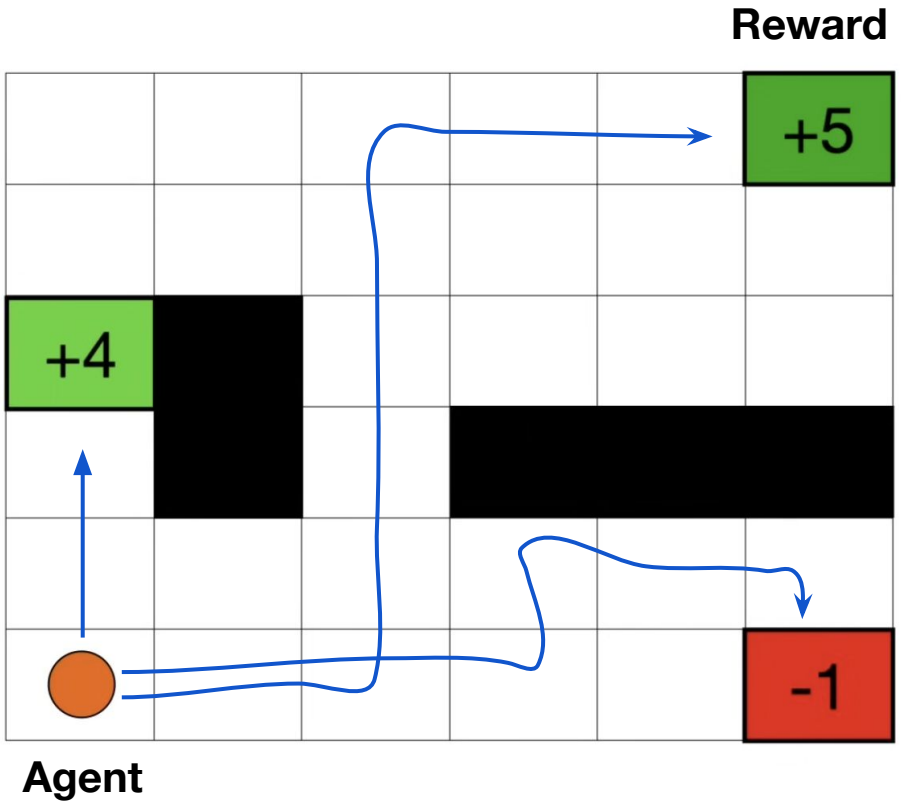
What is RL?

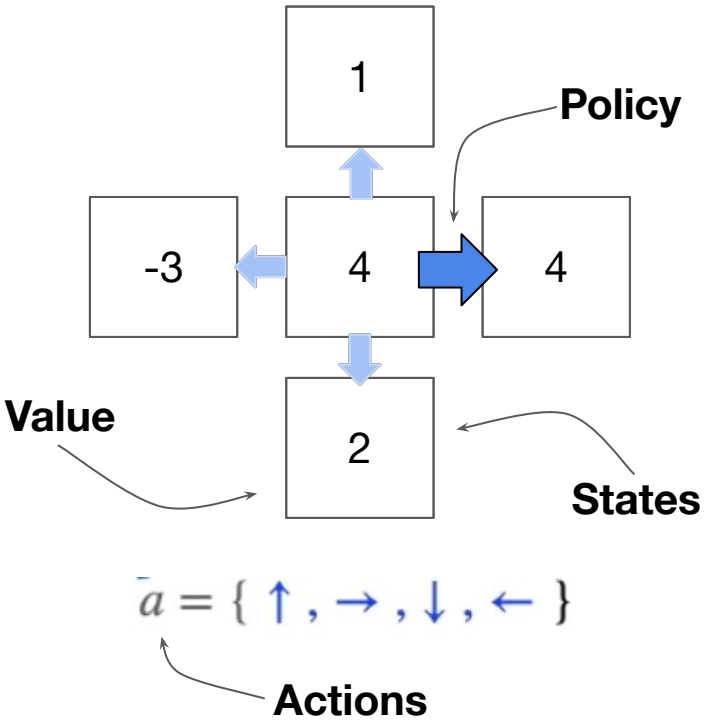
A very brief introduction

What is Reinforcement Learning (RL)?

- RL is a type of machine learning where we have an
  - **environment X**
  - ability to make **actions A**
  - get a delayed **reward R**
- An agent learns by interacting with an environment.
- The agent receives rewards for good actions and penalties for bad ones.
- The goal is to maximize cumulative rewards over time.

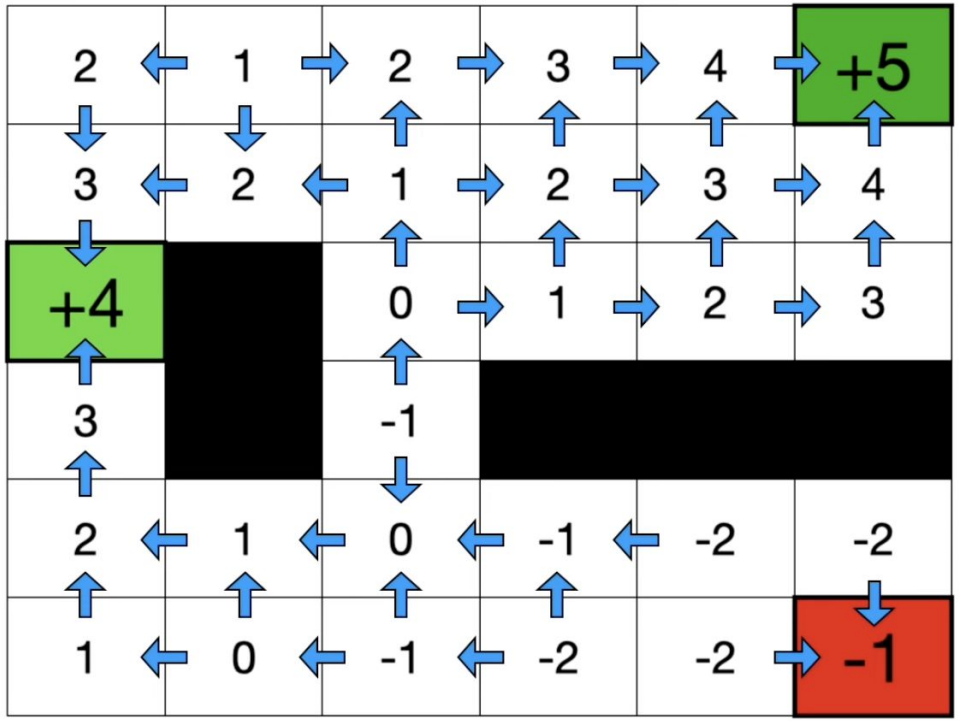
GridWorld

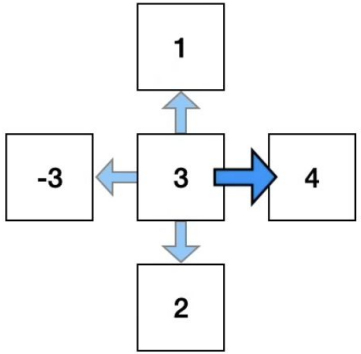




Source: https://www.youtube.com/watch?v=SgC6AZss478&list=PLs8w1Cdi-zvYviYYw_V3qe6SINReGF5M-&index=1

Good Policy: Where is the best place to move?





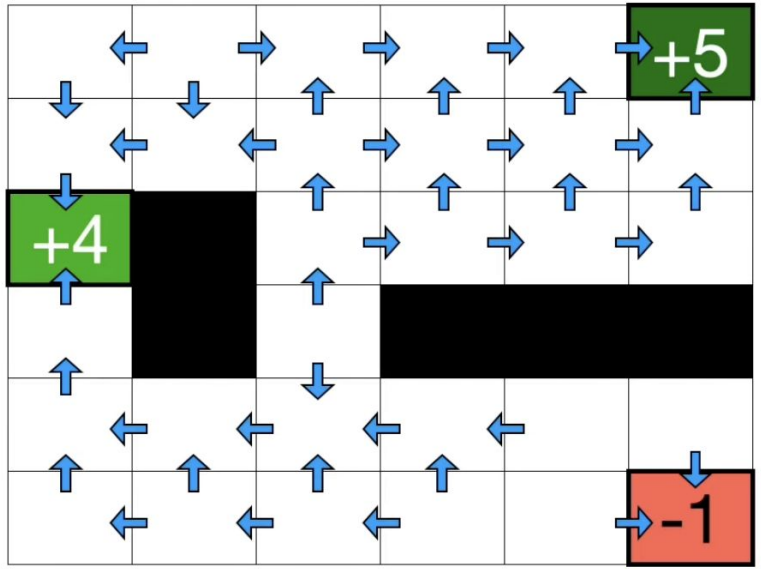
Source: https://www.youtube.com/watch?v=SgC6AZss478&list=PLs8w1Cdi-zvYviYYw_V3qe6SINReGF5M-&index=1

Optimal values and policy

Values

2	1	2	3	4	+5
3	2	1	2	3	4
+4		0	1	2	3
3		-1			
2	1	0	-1	-2	-2
1	0	-1	-2	-2	-1

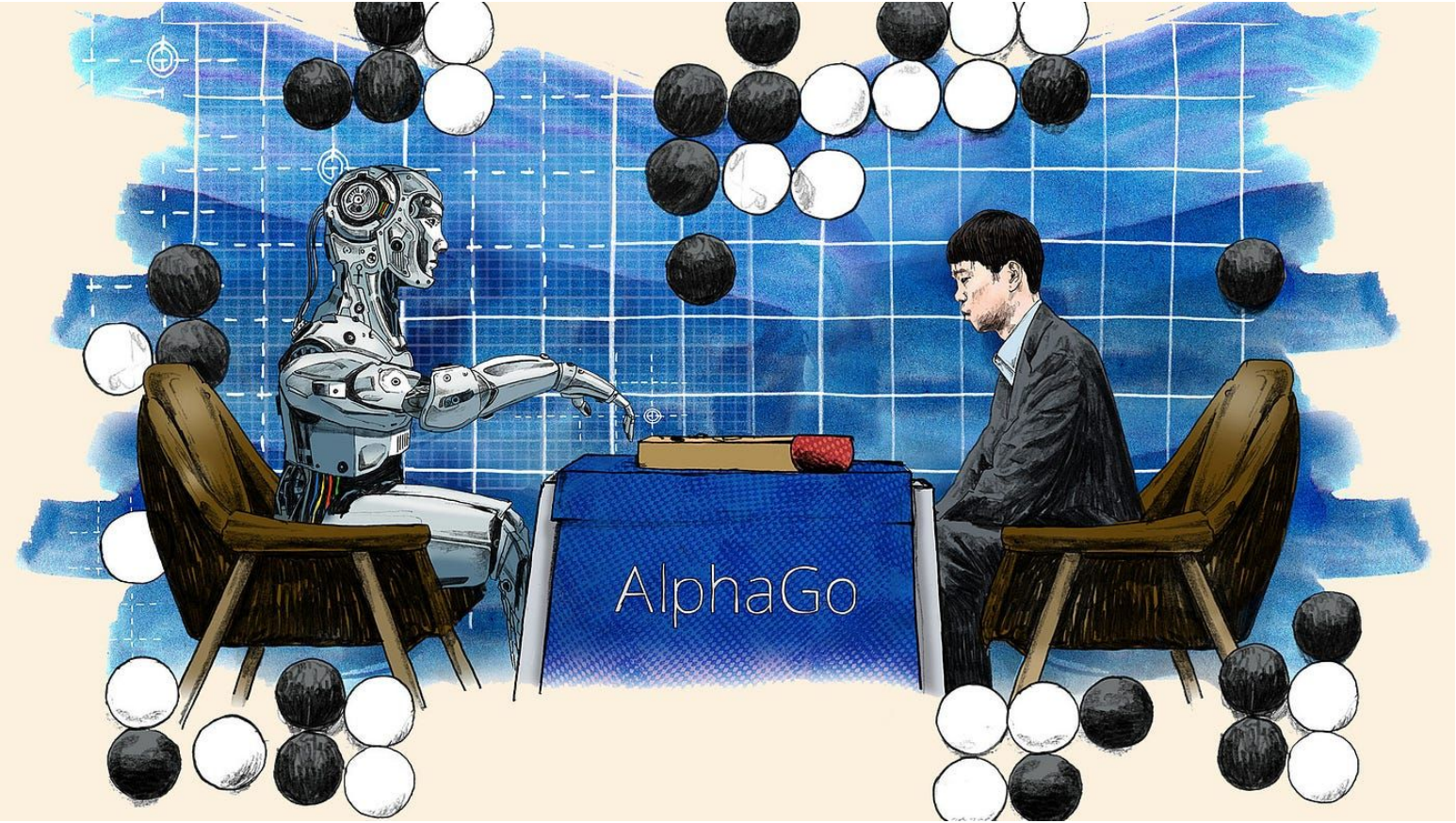
Policy



Source: https://www.youtube.com/watch?v=SgC6AZss478&list=PLs8w1Cdi-zvYviYYw_V3qe6SINReGF5M-&index=1

Why is RL Important?

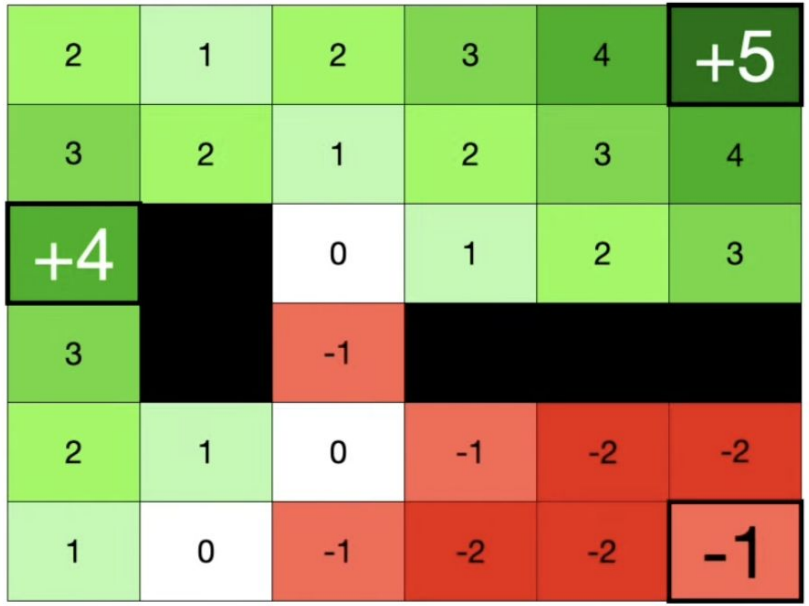
- Helps in optimizing long-term decision-making
- Used in game playing (AlphaGo, Dota 2 AI), robotics, autonomous driving, and chatbot optimization



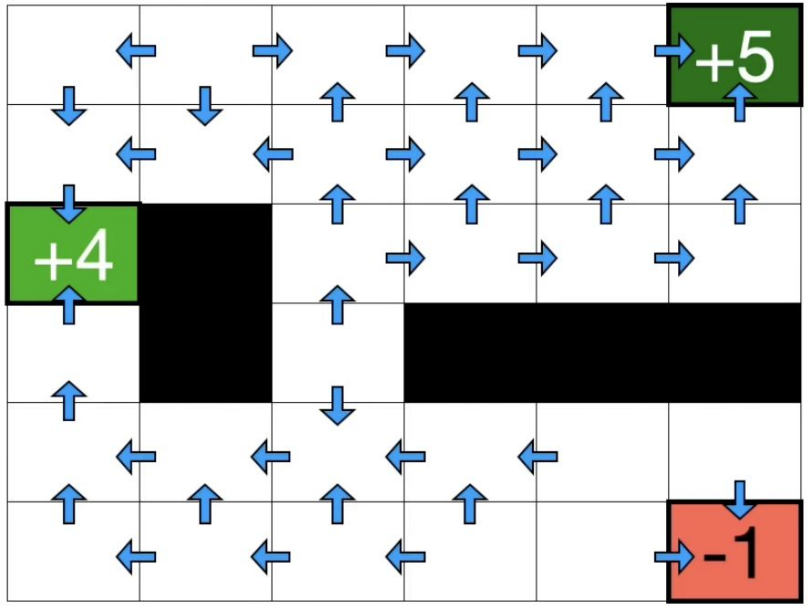
Img: https://ai.plainenglish.io/how-deepminds-alphago-became-the-world-s-top-go-player-5b275e553d6a

How can we learn these optimal values and policy?

Values

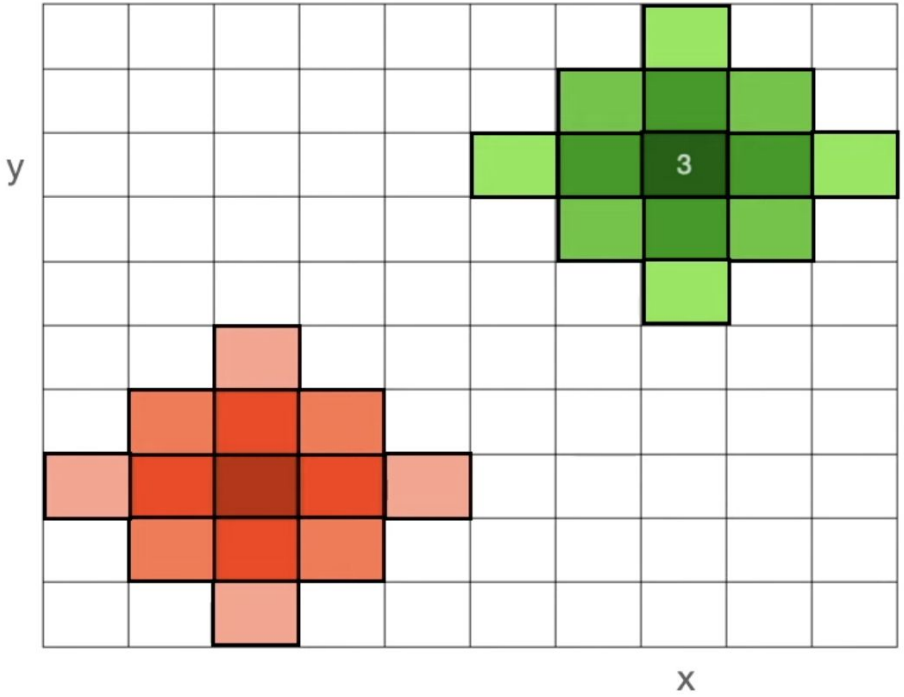


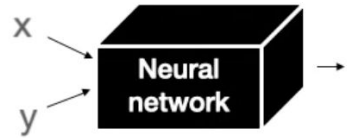
Policy

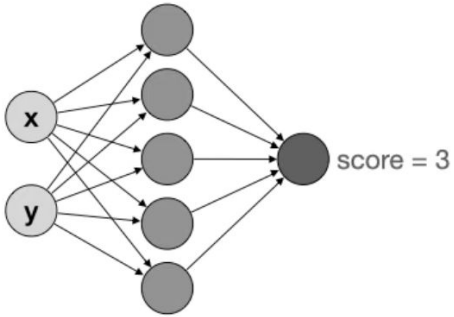


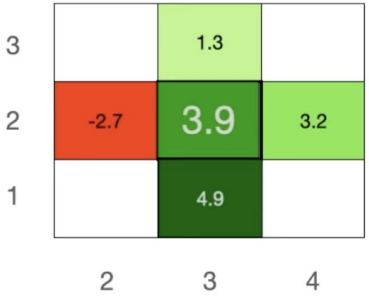
Source: https://www.youtube.com/watch?v=SgC6AZss478&list=PLs8w1Cdi-zvYviYYw_V3qe6SINReGF5M-&index=1

Value neural networks



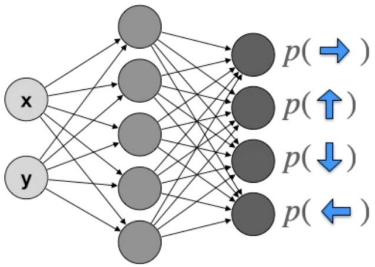


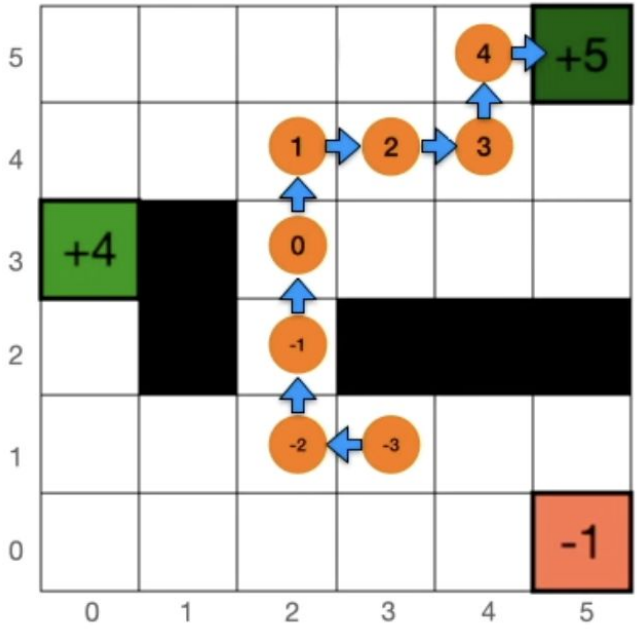




Source: https://www.youtube.com/watch?v=SgC6AZss478&list=PLs8w1Cdi-zvYviYYw_V3qe6SINReGF5M-&index=1

Policy neural networks

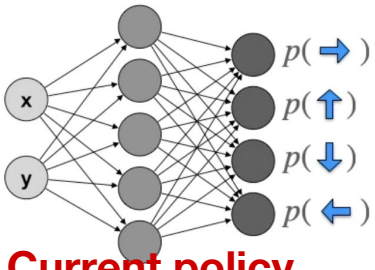




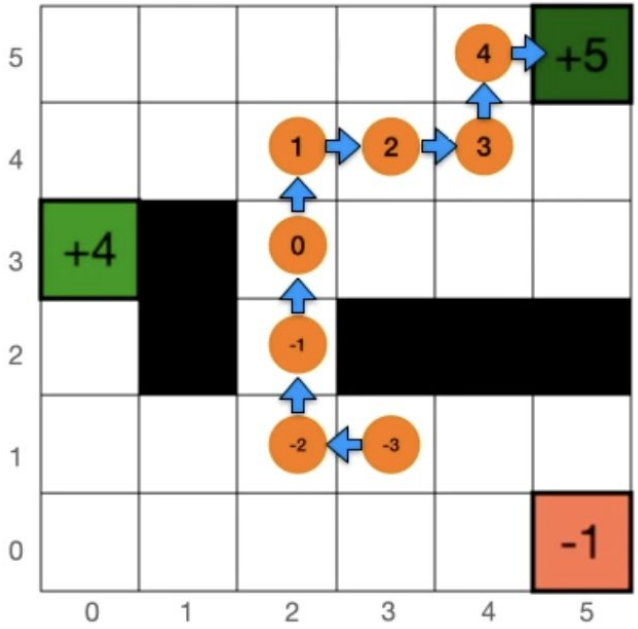
Gain	x	y	action	Probability	Increase log loss	change
4	4	5	→	$p(\rightarrow) = 0.3$	4 $\ln(p(\rightarrow))$	increase
3	4	4	↑	$p(\uparrow) = 0.9$	3 $\ln(p(\uparrow))$	increase
2	3	4	→	$p(\rightarrow) = 0.1$	2 $\ln(p(\rightarrow))$	increase
1	2	4	→	$p(\rightarrow) = 0.2$	1 $\ln(p(\rightarrow))$	increase
0	2	3	↑	$p(\uparrow) = 0.5$	0 $\ln(p(\uparrow))$	stay
-1	2	2	↑	$p(\uparrow) = 0.4$	-1 $\ln(p(\uparrow))$	decrease
-2	2	1	↑	$p(\uparrow) = 0.3$	-2 $\ln(p(\uparrow))$	decrease
-3	3	1	←	$p(\leftarrow) = 0.7$	-3 $\ln(p(\leftarrow))$	decrease

Source: https://www.youtube.com/watch?v=SgC6AZss478&list=PLs8w1Cdi-zvYviYYw_V3qe6SINReGF5M-&index=1

Policy neural networks



Current policy

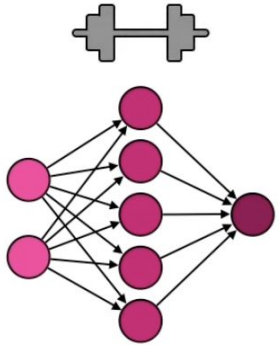


Gain	x	y	action	Probability	Increase log loss	change

Source: https://www.youtube.com/watch?v=SgC6AZss478&list=PLs8w1Cdi-zvYviYYw_V3qe6SINReGF5M-&index=1

Value and Policy neural networks

Value neural network

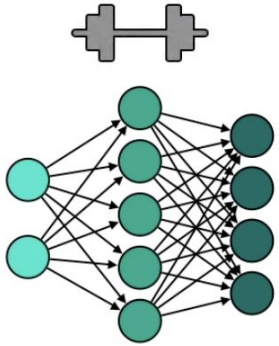


Input: state
Output: value

Approximates values

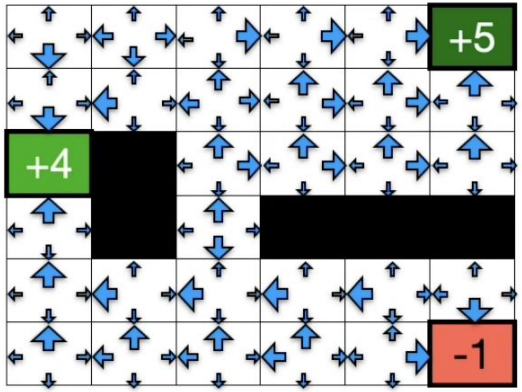
3.1	1.4	2.5	3.2	2.7	+5
2.2	-0.2	1.3	2.9	4.1	3.2
+4		-0.5	1.7	2.8	3.2
3.9		0.2			
2.6	1.5	1.1	0.3	-1.8	-2.3
1.4	0.8	0.1	-1.5	-2	-1

Policy neural network



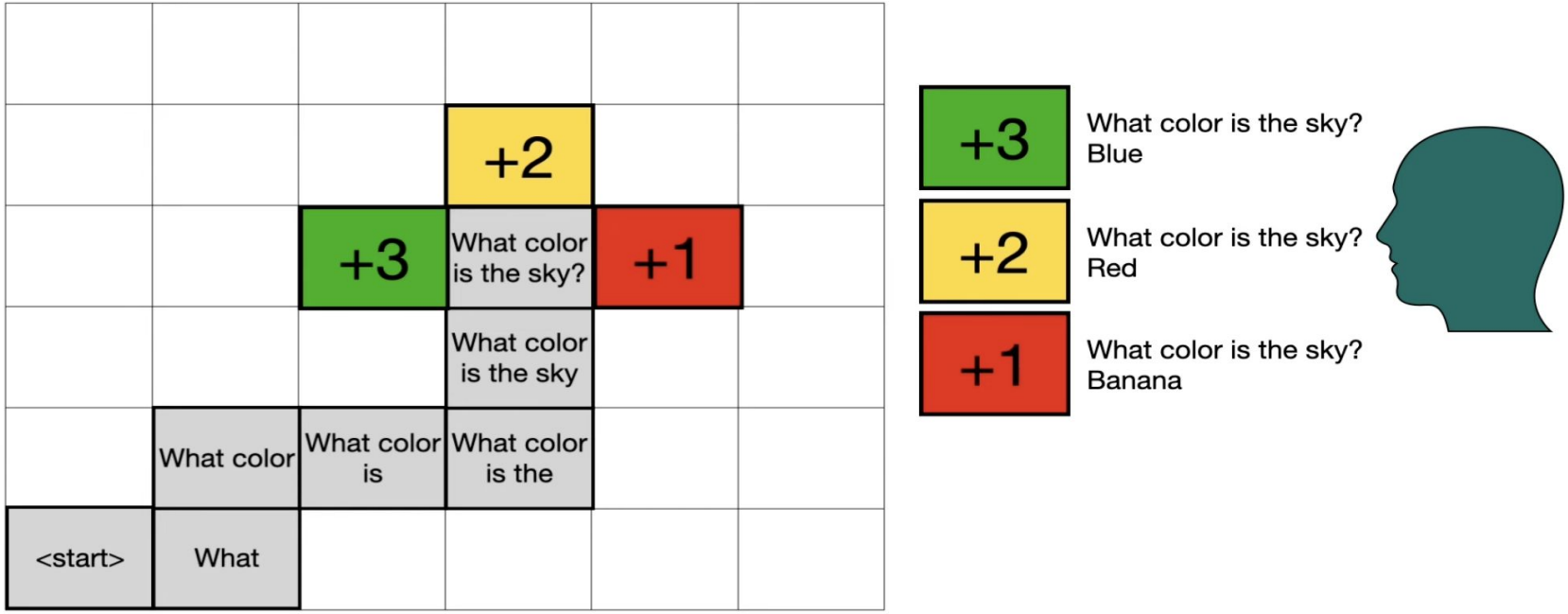
Input: state
Output: actions

Approximates policy



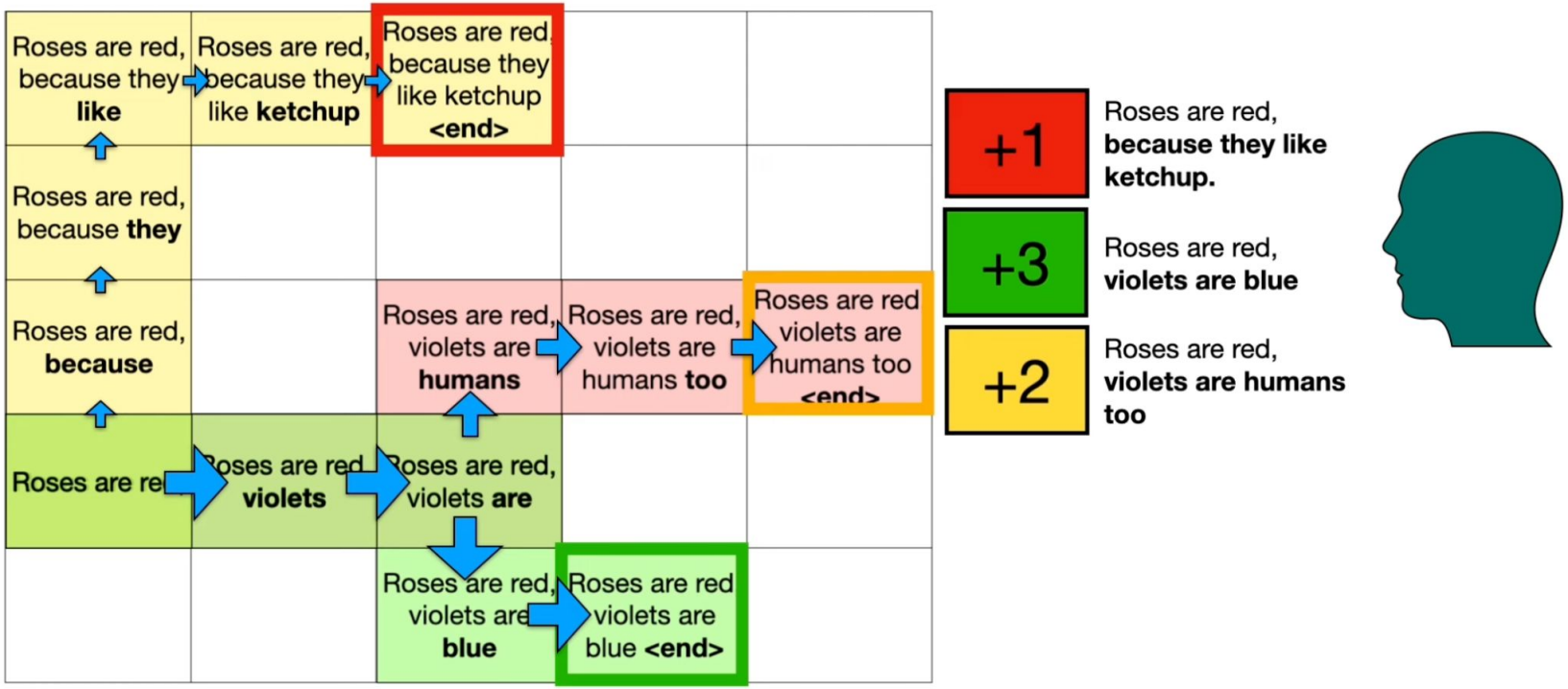
Source: https://www.youtube.com/watch?v=SgC6AZss478&list=PLs8w1Cdi-zvYviYYw_V3qe6SINReGF5M-&index=1

RL for language



Source: https://www.youtube.com/watch?v=Z_JUqJBpVOk&list=PLs8w1Cdi-zvYviYYw_V3qe6SINReGF5M-&index=4

RL for language

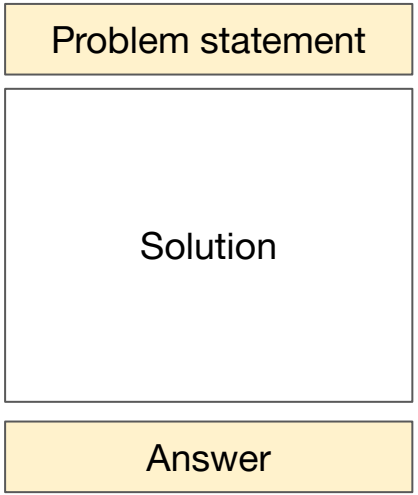


Source: https://www.youtube.com/watch?v=Z_JUqJBpVOk&list=PLs8w1Cdi-zvYviYYw_V3qe6SINReGF5M-&index=4

RL for Language Modeling

Why do we need RL?

We are given problem statement (prompt) and the final answer. We want to practice solutions that take us from problem statement to the answer, and “internalize” them into the model.



Which solution is better for inclusion in instructional data?

Emily buys 3 apples and 2 oranges. Each orange costs $2. The total cost of all the fruit is $13. What is the cost of each apple?

Answer: $3

—

Set up a system of equations.

X = price of apples

3*x + 2*2 = 13

3*x + 4 = 13

3*x = 9

X = 3

—

13 - 4 = 9, 9/3 = 3

—

The oranges cost 2*2 = 4.

So the apples cost 13 - 4 = 9.

There are 3 apples.

So each apple costs 9/3 = 3.

—

(13 - 4)/3 = 3.

ChatGPT 5 Share

Emily buys 3 apples and 2 oranges. Each orange costs $2. The total cost of all the fruit is $13. What is the cost of each apple?

Let's find the cost of each apple step by step carefully.

1️⃣ Let the cost of one apple be **x dollars**.

2️⃣ Each orange costs **$2**, and there are 2 oranges, so total orange cost = $2 \times 2 = 4$.

3️⃣ Total cost of all fruit is $13, so:

$$3x + 4 = 13$$

4️⃣ Subtract 4 from both sides:

$$3x = 9$$

5️⃣ Divide both sides by 3:

$$x = 3$$

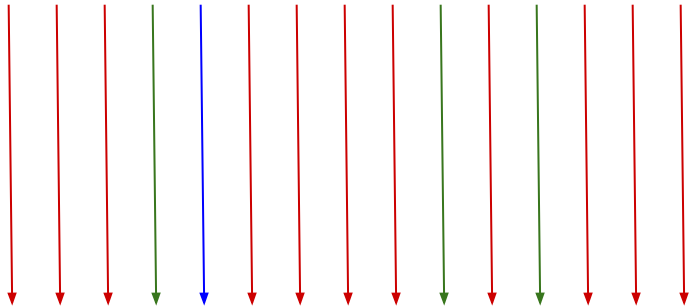
✅ **Each apple costs $3.**

Goals of RL:

- **Correctness: Obtain correct answers**
- Presentation: Effectively present results to humans

prompt

Emily buys 3 apples and 2 oranges. Each orange costs $2. The total cost of all the fruit is $13. What is the cost of each apple?



solutions

Answer: 3

We generate 15 solutions.

Only 4 of them got the right answer.

Take the top solution (each right and short).

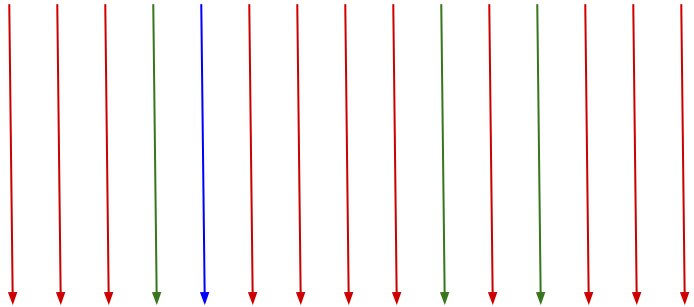
Train on it.

Repeat many, many times.

Verifiable vs un-verifiable domains

prompt

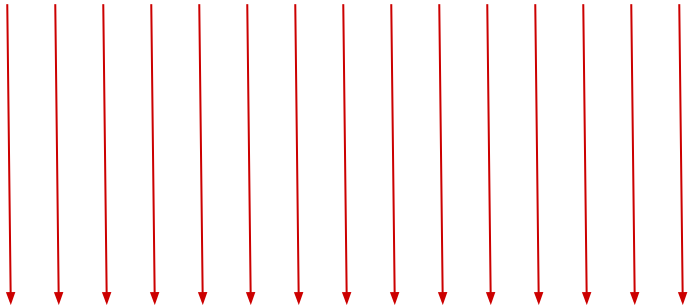
Emily buys 3 apples and 2 oranges. Each orange costs $2. The total cost of all the fruit is $13. What is the cost of each apple?



Answer: $3

prompt

Write a super funny joke about pelicans



Problem: how we do score theses?

Reinforcement Learning from Human Feedback (RLHF)

RLHF (Reinforcement Learning from Human Feedback)

Reinforcement Learning in un-verifiable domains

Fine-Tuning Language Models from Human Preferences

Daniel M. Ziegler* **Nisan Stiennon*** **Jeffrey Wu** **Tom B. Brown**
Alec Radford **Dario Amodei** **Paul Christiano** **Geoffrey Irving**
OpenAI
{dmz,nisan,jeffwu,tom,alec,damodei,paul,irving}@openai.com

Ziegler et al (OpenAI). 2019. Fine-tuning language models from human preferences. https://arxiv.org/abs/1909.08593

Training language models to follow instructions with human feedback

Abstract

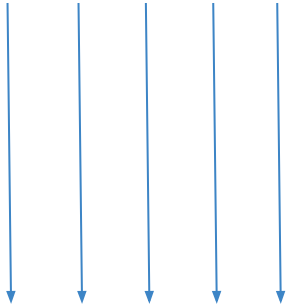
Making language models bigger does not inherently make them better at following a user's intent. For example, large language models can generate outputs that are untruthful, toxic, or simply not helpful to the user. In other words, these models are not *aligned* with their users. In this paper, we show an avenue for aligning language models with user intent on a wide range of tasks by fine-tuning with human feedback. Starting with a set of labeler-written prompts and prompts

Ouyang et al. (OpenAI) 2022. Training language models to follow instructions with human feedback. In NeurIPS. https://arxiv.org/abs/2203.02155

If using a typical RL approach

prompt

“Write a super funny joke about pelicans”

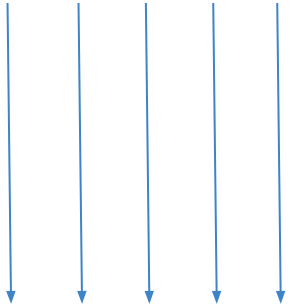


- Native approach
  - Run RL as usual, of 1,000 updates of 1,000 prompts of 1,000 rollouts (cost: 1,000,000,000 scores from humans)

If using an RLHF

prompt

“Write a super funny joke about pelicans”



reward model scores:	0.1	0.8	0.3	0.4	0.5
human ordering:	2	1	3	5	4

- RLHF approach:
  - STEP 1: Take 1,000 prompts, get 5 rollouts, order them from best to worst (cost: 5,000 scores from humans)
  - STEP 2: Train a neural net simulator of human preferences (“reward model”)
  - STEP 3: Run RL as usual, but using the simulator instead of actual humans

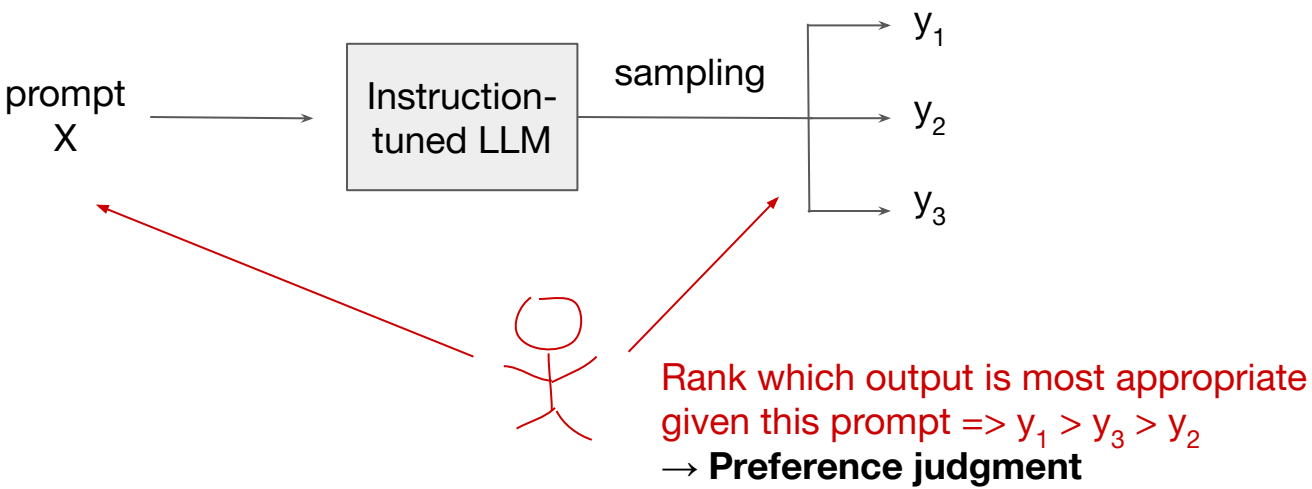
Human feedback

- **Fluency**: how natural is the output
- **Adequacy**: in translation, how well does the output reflect the input semantics?
- **Coherence**: does the output fit coherently in a discourse?
- **Truthfulness**: does the output contain accurate information and doesn’t mislead the user?
- **Helpfulness**: how effectively does the output address the user's needs or task?
- **Harmlessness**: does the output avoid causing harm, offense, or misinformation?
- etc. etc.

RLHF: Aligning LLMs with human intents

- Start with a large pre-trained LLM
- Step 1: instruction-tuning
- Limitations of Instruction-Tuning
  - The model does not learn from negative feedback.
  - Tasks like open-ended creative generation have no right answer.
    - They allow multiple valid responses, but training data often includes only one.
  - The model struggles to recognize when it should say, "I don't know."
  - The training process does not optimize for user-specific needs or values (human preferences).
    - Language modeling penalizes all token-level mistakes equally, but some errors are worse than others.

Human feedback: Preference ratings



- Limitation of human feedback: extremely expensive to collect

Reward model

- Idea: Can we train a model to predict human preference judgement?
- Reward Model
  - Input: prompt x, output y;
  - Output: A scalar score representing preference
- One way to train this model is using the **Bradley-Terry pairwise preference model**.

Bradley-Terry pairwise preference model

- x is the input (e.g., a user prompt)
- y_W is the preferred response
- y_L is the less preferred response
- $r(x, y)$ is the reward function assigning a scalar score
- The probability that **response** y_W **is preferred over** y_L in terms of their reward scores:

$$P(y_W > y_L \mid x) = \frac{e^{r(x,y_W)}}{e^{r(x,y_W)} + e^{r(x,y_L)}}$$

If $r(x,y_W) \gg r(x,y_L)$, the probability approaches 1.

If $r(x,y_W) \approx r(x,y_L)$, , the probability is close to 0.5.

If $r(x,y_W) \ll r(x,y_L)$, , the probability approaches 0.

- The negative log-likelihood (NLL) loss of this probability is defined as:

$$\mathcal{L} = -\log \sigma(r(x, y_W) - r(x, y_L))$$

Deriving the loss function for a preference model

$$P(y_W > y_L \mid x) = \frac{e^{r(x,y_W)}}{e^{r(x,y_W)} + e^{r(x,y_L)}}$$

Factor out $e^{r(x,y_W)}$ from the denominator

$$P(y_W > y_L \mid x) = \frac{1}{1 + e^{r(x,y_L) - r(x,y_W)}}$$

Recognizing that this matches the sigmoid function: $\sigma(z) = \frac{1}{1+e^{-z}}$

$$\sigma(r(x, y_W) - r(x, y_L)) = \frac{1}{1 + e^{-(r(x,y_W) - r(x,y_L))}}$$

$$P(y_W > y_L \mid x) = \sigma(r(x, y_W) - r(x, y_L))$$

Thus, the probability of preference follows a sigmoid distribution of the score difference.

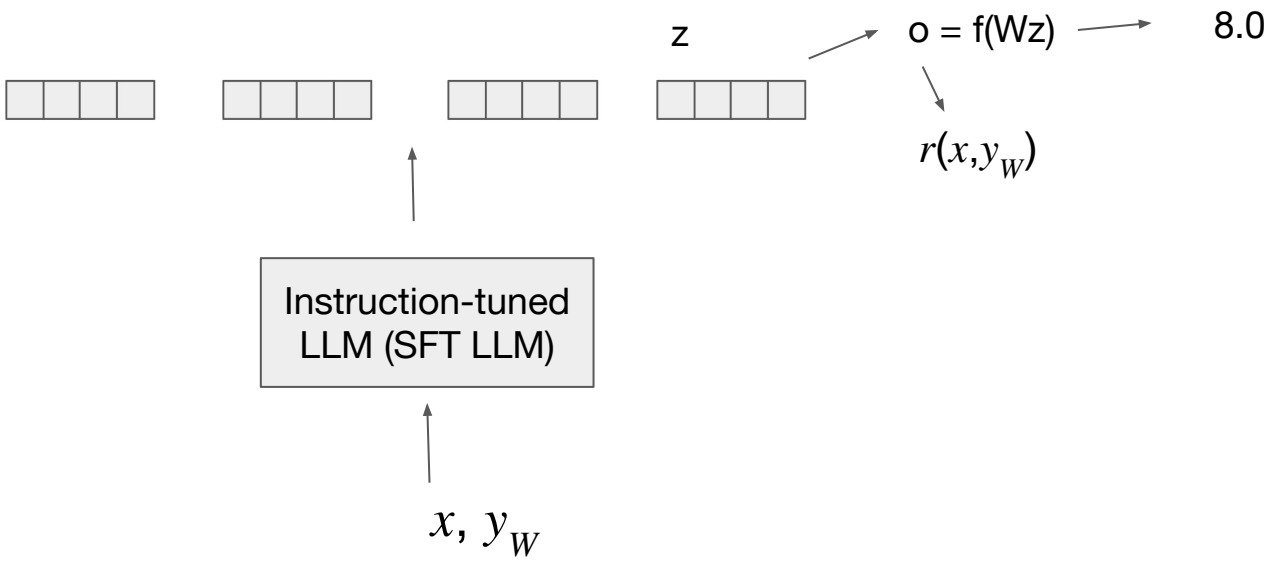
Since we want to **maximize the likelihood** that y_W is preferred over y_L, we take the **negative log-likelihood** to get a loss function:

$$\mathcal{L} = -\log P(y_W > y_L \mid x)$$

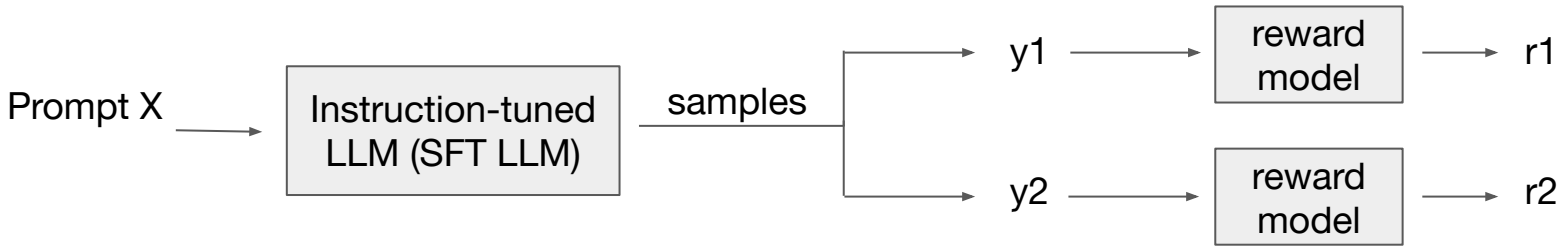
$$= -\log \sigma(r(x, y_W) - r(x, y_L))$$

Training the reward model

Intuition: good sample y_W's reward should be greater than y_L



Using the reward model



Aligning LLMs to human preferences using the reward model

- "best-of-n" sampling (rejection sampling)
  - Generate *n* samples for a given prompt, score each sample with the reward model, choose the sample with the highest reward
    - → computationally very expensive
- Fine-tuning to maximize $p(y_W \mid x)$
  - Generate a large dataset using reward model and finetune it
    - → tends to fail to learn from negative samples
- Use **reinforcement learning** to increase $p(y_W \mid x)$ slightly and decrease $p(y_L \mid x)$ slightly, where the amounts are functions of $r(x,y_W)$, $r(x, y_L)$
    - → Allows for fine-grained control over preference alignment

RLHF optimization

- We observe a reward only after generating a complete sequence.
- RL propagates the final reward to intermediate steps, guiding the model toward the optimal token sequence.
- The policy (π) refers to the probability distribution over token sequences that the model generates.

$\pi_{\text{ref}}(y \mid x) \rightarrow$ SFT checkpoint (the reference model)

$\pi(y \mid x) \rightarrow$ current policy model (initialized to π_{ref})

RLHF optimization objective

Maximizes the rewards

Prevents the model from changing too drastically

$$\pi^* = \max_{\pi} \mathbb{E}_{x \sim D, y \sim \pi} \left[r(x, y) - \beta D_{KL}(\pi(y \mid x) \parallel \pi_{\text{ref}}(y \mid x)) \right]$$

$$D_{KL}(\pi \parallel \pi_{\text{ref}}) = \sum_i \log \frac{\pi(w_i \mid w_1, \ldots, w_{i-1}, x)}{\pi_{\text{ref}}(w_i \mid w_1, \ldots, w_{i-1}, x)}$$

π^* is the optimized policy model.

$r(x, y)$ is the reward function for generating output y given input x.

$\pi(y \mid x)$ is the probability distribution of the current policy model.

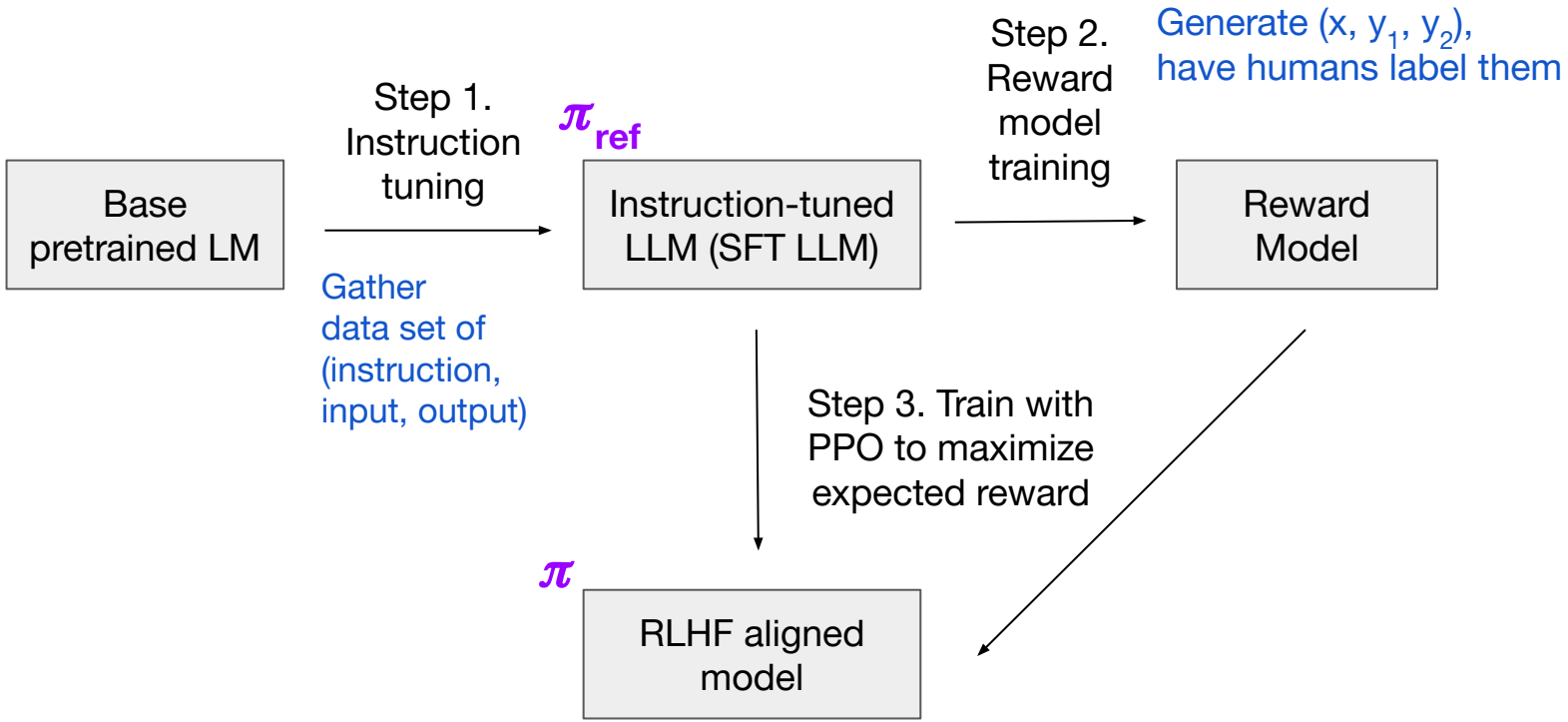
$\pi_{\text{ref}}(y \mid x)$ is the probability distribution of the SFT checkpoint (the reference model).

$D_{KL}(\pi \,//\, \pi_{\text{ref}})$ is the Kullback-Leibler (KL) divergence term to regularize policy updates.

β is a hyperparameter that controls the regularization strength.

- Optimize using Proximal Policy Optimization Algorithms (PPO) (Schulman et al. 2016) or REINFORCE (Williams et al. 1992)

RLHF pipeline



Any Questions?